LAB 5: STATIC CODE ANALYSIS

Name : Rushikesh

SRN : PES1UG24AM394

1. Objective

To improve the quality, security and style of the assigned Python program using static analysis tools. Pylint assesses code quality, Bandit identifies common security risks, and Flake8 checks style and formatting. Reports were generated before and after the code changes.

2. Program and Tools

The assigned program, expense_tracker.py, is a personal monthly expense tracker. It records expenses by category, rejects invalid amounts, totals spending per category, tracks the remaining budget, finds the biggest expense and checks a bank-sync password. The tools were run from the command line on the original and the corrected file.

Tool	Purpose
Pylint	Code-quality diagnostics: naming, documentation, programming-practice checks and an overall score.
Bandit	Checks for common insecure Python coding patterns.
Flake8	Checks formatting and style conventions (PEP 8), including whitespace and line length.

3. Issue Documentation and Fixes

Line numbers refer to the original expense_tracker.py; they differ in the corrected version.

Issue	Type	Original line(s)	Description	Fix applied
OS command injection via os.system (Bandit B605, High)	Security	61	os.system("xdg-open " + path) runs through a shell, so a crafted path can execute arbitrary commands.	Removed the shell. The path is resolved, checked to be a real file, and opened with webbrowser.open(file_uri).
Insecure temporary file, mktemp (Bandit B306, Medium)	Security	55	tempfile.mktemp() only returns a name, so another process can create or symlink it first (race condition).	Uses tempfile.mkstemp(), which creates the file atomically.
Hard-coded /tmp directory (Bandit B108, Medium)	Security	8	REPORT_DIR = "/tmp/expense_reports" is a predictable, world-writable location.	Built from tempfile.gettempdir(); folder created with mode 0o700.
Hard-coded password (Bandit B105, Low)	Security	5, 49-50	"demo-sync-pass" is stored in source and compared with ==.	Read from the BANK_SYNC_PASSWORD environment variable and compared with hmac.compare_digest.
Broad except	Quality	39-42	except Exception hides real bugs.	Removed; max(..., default=None) handles the empty case.
Global statement	Maintainability	6-7, 28-30, 33-35	Module-level expenses and monthly_budget changed via global.	Moved into an ExpenseTracker class as instance state.
range(len()) loop	Style	45-46	Iterating by index is verbose and error-prone.	Uses enumerate(..., start=1).
Function and variable naming	Style	Multiple	camelCase names (addExpense, setBudget, reportPath, ...) break snake_case convention.	Renamed to add_expense, set_budget, report_path, etc.

Issue	Type	Original line(s)	Description	Fix applied
Missing docstrings	Documentation	1, 11-64	Module and functions lacked purpose descriptions.	Added module, class and function docstrings.
Formatting violations (E203, E302, E261, E501)	Style	15, 44, 74, 77	Flake8 reported spacing, blank-line and line-length violations.	Adjusted spacing and wrapped the long line.
Unguarded main() call	Quality	80	The demo ran on import.	Wrapped in if __name__ == "__main__".

4. Before-and-After Results

Tool	Before	After	Outcome
Pylint	5.27/10 (26 messages)	10.00/10	Target above 9.5 achieved.
Bandit	Four findings: 1 High, 2 Medium, 1 Low	No issues identified	All reported findings addressed.
Flake8	Four style violations	No output / no violations	Formatting findings addressed.

The program output is identical before and after the changes (program_output.txt in before/ and after/). Evidence files: pylint, bandit and flake8 reports in the before/ and after/ folders.

5. Reflection

5.1 What is static code analysis?

Static code analysis examines source code without executing it to identify potential bugs, security vulnerabilities and style violations. It helps developers detect issues early.

5.2 What are the purposes of Pylint, Bandit, and Flake8?

Pylint reports code-quality and programming-practice issues and gives a score. Bandit identifies common insecure coding patterns. Flake8 checks Python style and formatting, including whitespace and line length.

5.3 What improvements were achieved?

The Pylint score improved from 5.27/10 to 10.00/10. Bandit reported no issues after the changes (down from four), and Flake8 produced no violations. Changes included removing shell execution, safer temporary-file handling, moving the password out of the source, removing the broad exception and global state, snake_case names and docstrings.

5.4 Why is static analysis useful?

It finds defects and risky patterns before code is deployed. It improves readability, maintainability, consistency and security, and provides repeatable checks after every code change.

5.5 What did this lab demonstrate?

The tools complement one another: Pylint focuses on broad code quality, Bandit on security, and Flake8 on style. Running them before and after the changes provides measurable verification that the code improved without changing its behavior.